Week 1 Summary: Netflix's AWS Migration and Chaos Engineering

Group 16 | Prince Chovatiya (202301067) · Ayush Patel (202301084) · Rishit Raj Jain (202301167)

Project: "Mini-Netflix: migrate, then break it on purpose" | **Theme:** Migration and Transformation (primary), Scalability (secondary)

1. Problem

- **Incident:** in Aug 2008 a database corruption in Netflix's own data center stopped DVD shipping for about 3 days [R1, R2].
- **Why it mattered:** the business was moving to streaming, which has to be up all the time. The system was a vertically scaled monolith on Oracle in one data center, so every layer was a single point of failure, and capacity had to be bought months ahead for unpredictable demand.

2. Decision

- Move to AWS and rebuild as distributed microservices instead of lifting the monolith, to add capacity in minutes and avoid large capital purchases [R1].
- Migration took about 7 years and finished around Jan 2016 when billing, the last data center piece, was shut down. Customer-facing services moved first, sensitive systems last [R3].
- Culture changed too: teams owned their services end to end ("you build it, you run it") [R2].

3. Architecture changes

Before	After	Why it helps
Monolith	Hundreds of microservices	A failure or deploy hits one service, not the whole site
Oracle (single relational DB)	Cassandra (NoSQL), later DynamoDB	No single DB host to lose; writes survive node failures
Own data center	EC2 Auto Scaling Groups behind ELB, S3, multi-region (US-East, US-West, EU)	Elastic capacity; a region can be evacuated
Third-party CDN / own delivery	Open Connect (own CDN inside ISPs)	Lower transit cost for video

Open-sourced supporting tools: Eureka (service discovery), Zuul (edge gateway), Hystrix (circuit breaker), Spinnaker (continuous delivery) [R10].

4. Resilience practices

- **Chaos Monkey (2010-11):** randomly kills production instances in business hours so services must survive instance loss [R4, R6].
- **Simian Army:** Latency Monkey (injects delay), Conformity Monkey (checks best practice), Chaos Gorilla (an Availability Zone), Chaos Kong (a whole region) [R4].
- **ChAP and FIT:** send a small slice of real traffic to an experiment group and compare with a control group to limit blast radius; regional failover drills [R7].
- **Principles of Chaos:** define steady state, hypothesise it holds, inject real-world failures, try to disprove, minimise blast radius [R8, R9].

All.

5. Outcomes and cost angle

- Higher availability (losing an instance, AZ or, in drills, a region does not take the service down), elastic scaling for peaks, and global expansion. Engineers design for failure from the start.
- Trade-off: microservices add complexity (tracing, many pipelines, harder debugging), which Netflix paid for with heavy tooling.
- Netflix's stated reasons were elasticity, speed and fewer large capital purchases, not lower unit cost [R1]. Exact savings are not public, so we model cost ourselves (our own estimate, Week 3).
- Netflix still runs its own CDN (Open Connect): at high, steady video volume, owning delivery is cheaper than renting. This is a counter-example to "cloud is always cheaper" [R5].

6. Our project and what we add beyond the case

Idea A chosen. Stages: S0 baseline monolith in Docker; S1 lift-and-shift to one EC2 VM; S2 refactor to services + ALB + Auto Scaling + managed DB; S3 chaos (boto3 script and AWS FIS); S4 compare S1 vs S2 under the same chaos; S5 cost model (AWS vs GCP vs on-prem), reported as cost per stage and per 1,000 requests.

New technology: Netflix's 2010-16 tooling is partly obsolete, so we reproduce the ideas with current tools.

Addition	What we do	Ref

AWS Fault Injection Service	Managed terminate/latency experiments with stop conditions, compared with our hand-written boto3 "mini Chaos Monkey"	R12
Terraform / CloudFormation	Build S1 and S2 as code: reproducible, measurable setup effort, easy teardown to save money	R14
Prometheus + Grafana, CloudWatch	Dashboards for latency, error rate and instance count during chaos (steady-state metrics)	R8, R15
Circuit breaker, retry, graceful degradation	Reproduce Hystrix: if recommendations is down, catalogue returns a popular list; test with and without	R10, R16
SLO and Well-Architected review	Set an availability target (e.g. 99.9%) and score S1 and S2 against the Reliability Pillar	R16, R18
Stretch goals	Containers on ECS Fargate/EKS; Lambda for one service; Spot vs On-Demand pricing	R13, R17, R19

Core additions: Terraform, Prometheus/Grafana, circuit breaker and FIS. The rest are stretch goals.

7. References

R1 Netflix Tech Blog, "Four Reasons We Choose Amazon's Cloud as Our Computing Platform" (2010)

R2 Netflix Tech Blog, "5 Lessons We've Learned Using AWS" (2010)

R3 Netflix Tech Blog, "Completing the Netflix Cloud Migration" (Feb 2016)

R4 Netflix Tech Blog, "The Netflix Simian Army" (2011)

R5 Netflix Open Connect (openconnect.netflix.com)

R6 GitHub Netflix/chaosmonkey; "Chaos Monkey Released Into The Wild" (2012)

R7 Netflix Tech Blog, "ChAP: Chaos Automation Platform"; "FIT: Failure Injection Testing"

R8 principlesofchaos.org

R9 Rosenthal, Jones et al., *Chaos Engineering* (O'Reilly, 2020)

R10 Netflix OSS on GitHub: Eureka, Zuul, Hystrix, Spinnaker

R11 Netflix investor letters and 10-K filings

R12 AWS Fault Injection Service documentation

R13 AWS docs: ECS on Fargate / EKS

R14 Terraform AWS provider docs / CloudFormation user guide

R15 Prometheus, Grafana and CloudWatch documentation

R16 AWS Well-Architected Framework, Reliability Pillar; Resilience4j docs

R17 AWS Lambda developer guide

R18 Beyer et al., *Site Reliability Engineering* (Google/O'Reilly, 2016)

R19 AWS and GCP pricing calculators; EC2 Spot and Savings Plans pages

R20 Basiri et al., "Chaos Engineering," *IEEE Software* 33(3), 2016

R21 Alvaro et al., "Automating Failure Testing Research at Internet Scale," ACM SoCC 2016

R22 DeCandia et al., "Dynamo: Amazon's Highly Available Key-value Store," SOSP 2007

R23 Lakshman and Malik, "Cassandra: A Decentralized Structured Storage System," 2010

R24 AWS Prescriptive Guidance, "Large migrations" and the 6 Rs

R25 Fowler and Lewis, "Microservices," martinfowler.com (2014)

R26 AWS re:Invent talks on Netflix architecture and resilience